

Project 1: Running ML Containers like JupyterLabs and MLFlow with Docker

Goal

Use pre-built AI/ML Docker images to run real-world tools like JupyterLabs and MLFlow. Learn how to connect, manage, and persist your work — with a full lifecycle workflow.

Lab: "Run JupyterLabs, MLFlow in Docker"

Requirements

- Docker Desktop / Rancher Desktop (installed and running)
 - Browser access (for Jupyter)
 - Basic terminal skills
-

Lab Contents

Part 1: Launch MLFlow using a Container

Step 1: Pull the Image

```
docker pull ghcr.io/mlflow/mlflow:latest
```



Step 2: Run with Port Mapping

```
docker run -p 5555:5000 ghcr.io/mlflow/mlflow:latest mlflow server --host 0.0.0.0
```

- Open in browser using <http://localhost:5555/>

While the MLFlow container is running and you are able to access it, on the console you are stuck attached with the container

[sample outout]

```
docker run -p 5555:5000 ghcr.io/mlflow/mlflow:latest mlflow server --host 0.0.0.0
[2025-05-26 04:23:01 +0000] [13] [INFO] Starting gunicorn 23.0.0
[2025-05-26 04:23:01 +0000] [13] [INFO] Listening at: http://0.0.0.0:5000 (13)
[2025-05-26 04:23:01 +0000] [13] [INFO] Using worker: sync
[2025-05-26 04:23:01 +0000] [14] [INFO] Booting worker with pid: 14
[2025-05-26 04:23:01 +0000] [15] [INFO] Booting worker with pid: 15
[2025-05-26 04:23:01 +0000] [16] [INFO] Booting worker with pid: 16
[2025-05-26 04:23:01 +0000] [17] [INFO] Booting worker with pid: 17
```

If you want come back to console you have to kill the container using `ctrl + c`. Try doing that.

Once exited, you can list it using

```
# this command will list only running containers
# you will not see mlflow running
docker ps

# this command shows you last run container, even if its stopped (exited)
docker ps -l

# more commands you can explore
docker ps -n 2
```

```
docker ps -a
```

#note the container id, which you will use to delete the container

now delete the container as

```
# replace xxxx with actual container id/name noted above
```

```
docker rm xxxx
```

```
# if you are removing a running container add -f option as
```

```
docker rm -f xxxx
```

You could instead launch the container in detached mode, which is the common way of running the container, which keeps running but in the background as ,

```
docker run -d -p 5555:5000 --name mlflow ghcr.io/mlflow/mlflow:latest mlflow  
server --host 0.0.0.0
```

where, newly added options are

- `-d` : run container in detached mode
- `--name mlflow` : sets the name of the container as `mlflow` instead of a auto generated random name

you can list the containers using

```
docker ps
```

and connect to it using <http://localhost:5555/> (replace localhost with actual hostname / ip if you have set up docker on a remote server/VM)

Few more container manageemnt commands that you could try

```
# check the logs for mlflow container
docker logs mlflow

# follow the logs. exit with ^c
docker logs -f mlflow

# Get inside container's shell with
docker exec -it mlflow sh

# use ^d to exit
```

◆ Part 2: Run Jupyter for ML Notebooks with a local Volume

Step 1: Create Local Directory

```
mkdir -p ~/ml-docker/notebooks
```

Step 2: Run with Volume Mount

```
docker run -d -p 8888:8888 --name notebook -v ~/ml-docker/notebooks:/home/
jovyan/work jupyter/scipy-notebook
```

- Check terminal logs for URL with token.

```
docker logs notebook
```

[sample output]

```
....
```

To access the server, open this file in a browser:

file:///home/jovyan/.local/share/jupyter/runtime/jpserver-7-open.html
Or copy and paste one of these URLs:

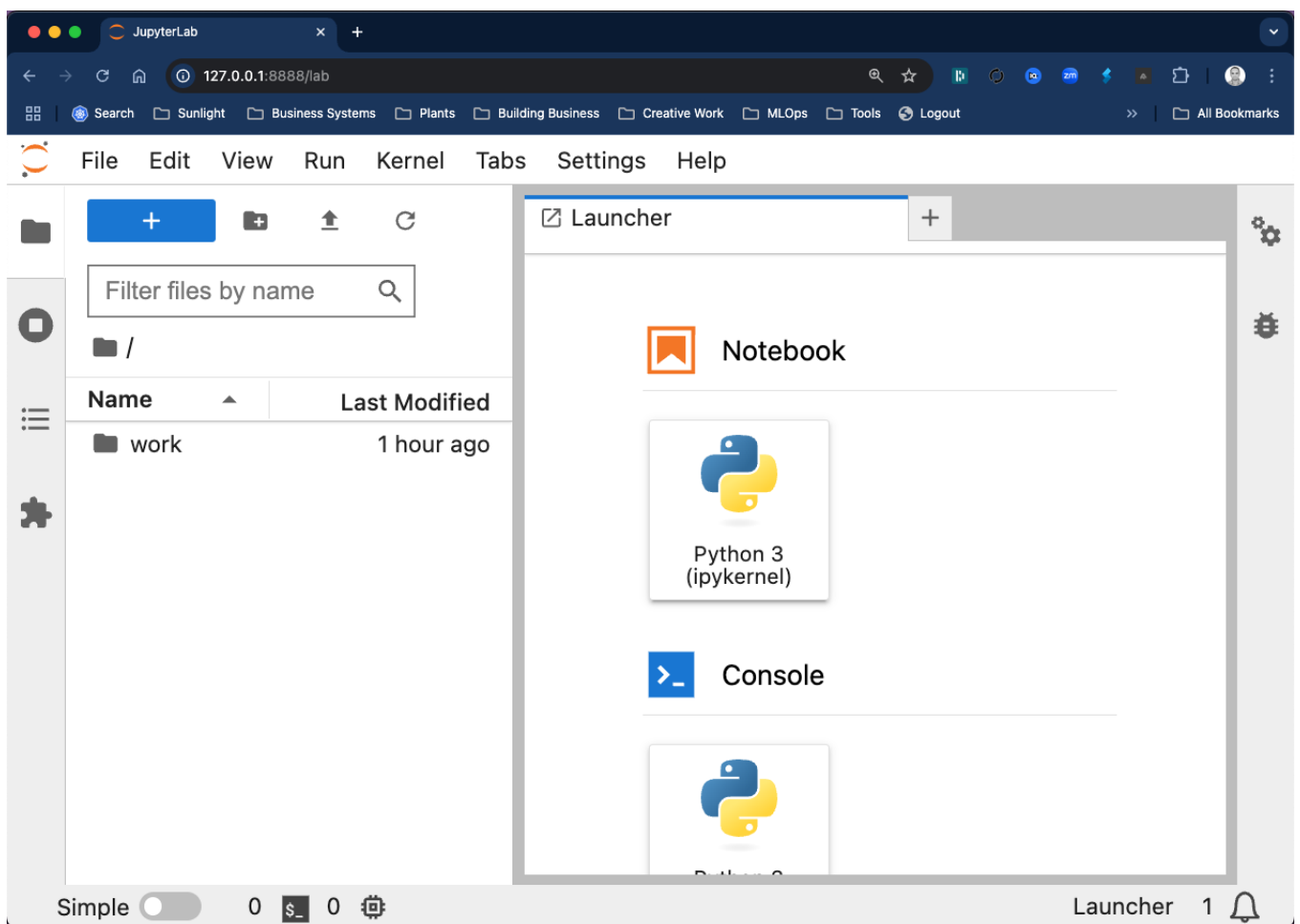
http://9a95b748605a:8888/lab?

token=4390ed0a681b70eaa3b87bd154fc786b833c712ca6ed24ff

http://127.0.0.1:8888/lab?

token=4390ed0a681b70eaa3b87bd154fc786b833c712ca6ed24ff

- Open in browser. e.g. <http://127.0.0.1:8888/lab>
- Create new notebook inside `work/`.



- Create a new notebook and save it. e.g. `work/basic-ml.ipynb`
- Open it from the host directory to check if it is shared via a volume between host and container.



Sample Notebook 1: Basic Pandas + Scikit-learn

Save this as `ml-docker/notebooks/basic-ml.ipynb` or create inside Jupyter.

```
# Step 0 / Cell 0
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, r2_score, mean_squared_error

# Step 1: Load the Iris dataset
print("📁 Loading the Iris dataset...")
data = load_iris()

# Step 2: Explore the dataset structure
print("\n📝 Feature names:", data.feature_names)
print("🎯 Target classes:", data.target_names)
print("📐 Data shape:", data.data.shape)

# Step 3: Create a DataFrame for exploration
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target
print("\n🔍 First 5 rows of the dataset:")
print(df.head())

# Step 4: Define features (X) and target (y)
X = df[data.feature_names]
y = df['target']

# Step 5: Train a Logistic Regression model
print("\n⚙️ Training Logistic Regression model...")
model = LogisticRegression(max_iter=200)
model.fit(X, y)

# Step 6: Make predictions
y_pred = model.predict(X)
```

```
# Step 7: Evaluate the model
accuracy = accuracy_score(y, y_pred)
print(f"\n🇩🇪 Accuracy Score: {accuracy:.2f}")

print("\n📄 Classification Report:")
print(classification_report(y, y_pred, target_names=data.target_names))

# Step 8 : Confusion Matrix Plot
cm = confusion_matrix(y, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=data.target_names,
            yticklabels=data.target_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("🔍 Confusion Matrix")
plt.show()
```

Source : [Sample Notebook Code](#)

Save and run this from the JupyterLabs Notebook. If you are new to ML, refer to the video lesson to understand how.

🧠 Where is load_iris() getting the data from?

📦 It comes from Scikit-learn's built-in datasets module.

```
from sklearn.datasets import load_iris
```

The function load_iris():

- Loads a **toy dataset** (Iris flower dataset) that is bundled **inside the** scikit-learn **Python package**.
- The data is **not fetched from the internet** — it's **local and immediately available** when you install scikit-learn.

📖 About the Iris Dataset

-  **150 samples** of iris flowers
-  4 features per sample:
 - sepal length
 - sepal width
 - petal length
 - petal width
-  Target: 3 classes of iris (setosa, versicolor, virginica)

Why This Is Useful in Your Docker Lab

- No need for an internet connection or external CSV files.
 - Zero setup — works out of the box inside the jupyter/scipy-notebook image because scikit-learn is pre-installed.
 - Perfect for demonstrating end-to-end ML workflows (load → train → predict) without external dependencies.
-

Part 3: Connect Jupyter with MLFlow to log Experiments to

Step 1: Install mlflow with pip

Create a new notebook `run_experiment.ipynb` and execute the following command as one of the cells.

```
!pip install mlflow
```

Step 2: Add Experiment with MLFlow Tracking

Create a new notebook `run_experiment.ipynb` and add the following code


```
import mlflow
from mlflow.models import infer_signature
from sklearn.linear_model import LinearRegression
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

# 1. Set tracking URI to local MLflow server
mlflow.set_tracking_uri("http://host.docker.internal:5555")
print("📡 Tracking to:", mlflow.get_tracking_uri())

# 2. Set experiment name (create if not exists)
mlflow.set_experiment("simple-linear-demo")

# 3. Create and log a run
with mlflow.start_run():
    # Generate toy regression data
    X, y = make_regression(n_samples=100, n_features=1, noise=10,
random_state=42)

    # Train model
    model = LinearRegression()
    model.fit(X, y)

    # Predict and evaluate
    y_pred = model.predict(X)
    mse = mean_squared_error(y, y_pred)

    # Infer model signature and input example
    signature = infer_signature(X, y_pred)
    input_example = X[:5] # A small batch as sample input

    # Log parameters and metrics
    mlflow.log_param("model_type", "LinearRegression")
    mlflow.log_metric("mse", mse)

    # Log model with signature and example
    mlflow.sklearn.log_model(
        model,
        artifact_path="model",
        signature=signature,
        input_example=input_example
    )
```

```
print(f"✅ Run logged with MSE: {mse:.2f}")
```

Source: [run_experiment.ipynb.md](#)

◆ Part 4: Docker Essentials for AI/ML

Concept	Explanation
Image	Blueprint for containers (e.g., <code>jupyter/scipy-notebook</code>)
Container	Running instance of an image
Tag	Versioned label for an image (e.g., <code>latest</code> , <code>2.4.1</code>)
Port Mapping (<code>-p</code>)	Connect host port to container port (e.g., <code>-p 8888:8888</code>)
Detached Mode (<code>-d</code>)	Run container in background
Interactive Terminal (<code>-it</code>)	Keeps terminal attached for CLI-based containers
Volume Mount (<code>-v</code>)	Bind-mount host dir into container

◆ Part 5: Container Lifecycle Management Commands

```
docker ps                # Show running containers
docker ps -a             # Show all containers
docker stop <container_id> # Stop container
docker start <container_id> # Restart a stopped container
docker rm <container_id>   # Remove container
docker logs <container_id> # View logs
docker exec -it <container_id> bash # Open interactive shell
```

Lab Outcome

You now know how to:

- Pull and run ML-ready containers
 - Mount notebooks for persistence
 - Interact with Python-based ML frameworks inside Docker
 - Manage containers like a pro
-

Next Module Preview:

Build Your Own ML Development Environment with a Dockerfile

-  Learn how to define custom containers with your own Python dependencies and notebooks!

`#courses/docker-aiml/lab`